

HCI compendium

IUM 2024/2025

References

- **Advances in Human-Computer Interaction**
H. Rex Hartson, D. Hix, edits.
Ablex Pub. corp., Norwood,
New Jersey, USA, 1992
- **Designing the User Interface**
Ben Shneiderman
3rd Edition, Addison-Wesley, 1999
- **Humane Interfaces**
Jef Raskin, Addison-Wesley, 2000.
- **Human-Computer Interaction**
Alan Dix, Janet Finlay, Gregory Abowd, Russell Beale,
Prentice Hall, 3rd Edition, 2004
- **The Encyclopedia of Human-Computer Interaction**
<https://www.interaction-design.org/literature/book/the-encyclopedia-of-human-computer-interaction-2nd-ed>
- **Nielsen Norman Group**
<https://www.nngroup.com/articles/>

interfaces, why?

- without interfaces computers would be useless
- human-computer interfaces are not even as carefully designed as computer-computer interfaces
- computer science has a definite role in the design of human-computer interfaces

interfaces, what **for**?

- creativity is very important in the design of user interfaces
- two sides of the design problem:
 - 1) computer science issues (essentially programming a chosen set of algorithms favouring human-program communication and control) and
 - 2) human issues (essentially exploiting and supporting user's skills)

formal approaches

- formal approaches help in abstracting the details and subtleties of how computers are used; interface features should not be looked at within particular applications: undo in an airline reservation system but in a wide domain - undo in an interactive system
- sometimes - as we proceed in the design of user interfaces for a class of applications (i.e. word processors) such applications have evolved - what has been used in the past and recent present may also be used in the near future on new applications
- we have abstracted **general rules** (low cognitive load, few basic icons, possibility of undoing all actions, backtracking availability, on-line help, etc.)

formalizing?

- formalizing has its detractors: are we abstracting the user **too** much?
- how do we capture his intentions, his expectations, his skills? To formalize, in this context, is a **question-raiser** more than an issue-solver
- documentation: saying what is implemented and implementing what is said: high level of vision on behalf of the designers, implementors and **documentors**
- user interface design: **complexities** of the human-program communication needs + peculiarities of human users: they should all be understood, modelled, taken care of, documented, tested, validated, refined...

interface benefits

- enjoyment - playing games, hacking programs, communicating with others, exploration of data, composition of music/poetry/essays, listening to music, drawing/sketching, studying, etc.
- enabling new skills - augmentation $\neq$ enabling monitoring in a better way is an empowerment, controlling a super-jet may only be done via a special interface for the pilot
- delegation - the computer is left to do repetitive, dangerous, critical jobs which the human would not do as smoothly (a negative case arises when the user delegates full responsibility on incomplete tasks)
- safety - remote control operations to avoid dangerous locations (coal mines, radioactive stations, etc.)

more benefits

- development - cultural enrichment through interactive teaching programs: arithmetic drill, road signals for car driving, language practice, etc.
- doing things at once - many different stages of a job were done in sequence, for each mistake corrections had to be done everywhere (as in a presentation, course preparation, etc.) a presentation manager (like Power Point) manages slides, notes, outlines together with timings, transitions, colours, backgrounds, etc.
- computing - what computers were born for! integrate, plot, enhance an image, prove a theorem, compute a spacecraft

interaction technology: a chronology

On/Off >'48

Data input >'55

Data input on request >'65

Graphical input via light pen, mouse >'72

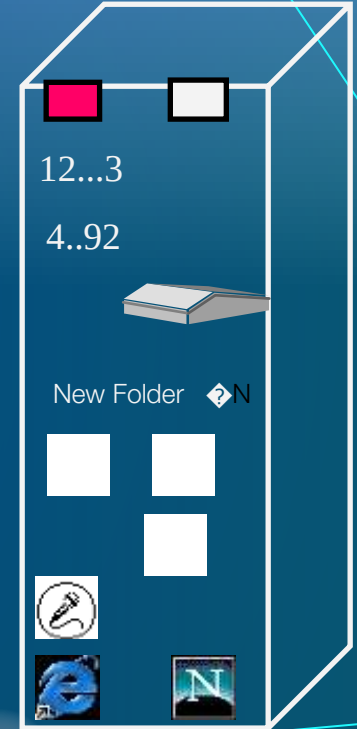
Menu selection >'78

Mac Style (desktop metaphor) >'83

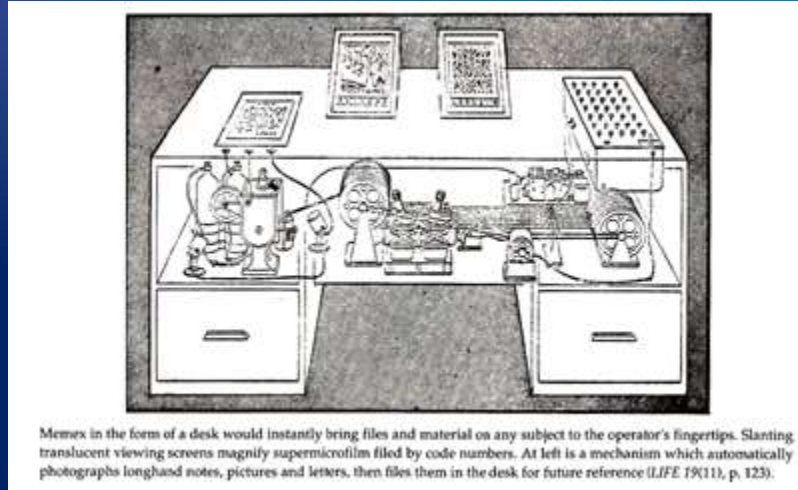
Multimedia tools >'90

Multimodal tools >'94

Internet tools & services > 00



Envisioning the future in 1945



A **Memex** is a device in which an individual stores all his books, records, and communications, and which is mechanized so that it may be consulted with exceeding speed and flexibility. It is an enlarged intimate supplement to his memory. (*Bush, 1945, 12*)

Douglas Engelbart

- Human Augmentation System
 - The Mother of All Demos: <https://www.youtube.com/watch?v=yJDv-zdhzMY>



NLS Mouse and
workstation



Ergonomic Keyboard
Console



First Mouse

Ivan Sutherland

→ The Ultimate Display – Ivan Sutherland

The ultimate display would, of course, be a room within which the computer can control the existence of matter. A chair displayed in such a room would be good enough to sit in. Handcuffs displayed in such a room would be confining, and a bullet displayed in such a room would be fatal. With appropriate programming such a display could literally be the Wonderland into which Alice walked. (Sutherland, 1965, 508)



Sketchpad

Large Scale Computing

- They were programmed using punch cards



Manual card punch machine



IBM card punch machines



These machines were also schemes that used the downtime of one user for another user who was currently active.
Courtesy IBM Corporate Archives.

Personal Computing

Desktop Computing

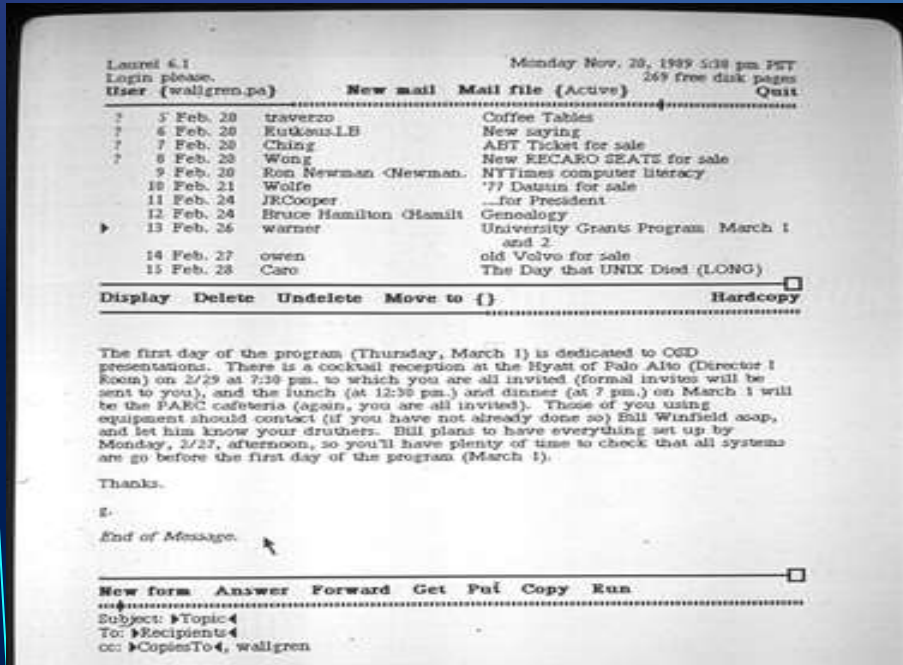


The Alto, developed at the Xerox Palo Alto Research Center in 1973, was the first computer to use a GUI that involved the desktop metaphor: pop-up menus, windows, and icons

The Xerox Alto computer (1973)

Courtesy Palo Alto Research Center.

Personal Computing



The Xerox Alto mail program (1973)



The Xerox Alto computer (1973)

Personal Computing

- Personal-Public Computing
 - Public Access Computing – The information divide
 - Public Information Appliances



Automated teller machine with
touchscreen.

Courtesy BigStockPhoto.com

Mobile Computing

- Desktop metaphors do not translate well to mobile devices.
- Hybrid desktop/mobile environments can afford optimal interaction efficiency.



Tablet computer



Cell phone



MP3 player

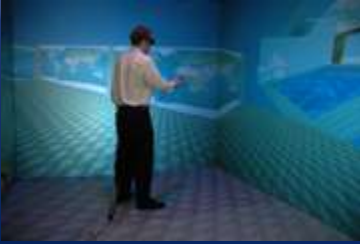


Laptop computer



On-board navigation system

Virtual Reality - *Immersive*



Sketching a virtual world in the VR design tool ShadowLight.

Photographs and ShadowLight application courtesy of Kalev Leetaru.

CAVE automated virtual environment at the National Center for Supercomputing Applications (NCSA).
<http://brighton.ncsa.uiuc.edu/~prajlich/cave.html>



Sensics piSight
Virtual Reality (VR) system.
<http://www.sensics.com/>



Meta Quest 3
<https://www.meta.com/it/quest/quest-3/>

Augmented Reality

- AR I/O devices

- Heads Up Displays (HUD)
 - Optical see through
 - Video see through



MicroOptical MD-6
Critical Data Viewer.
<http://microoptical.net/>



Sportvue MC1 motorcycle helmet
heads-up display.
<http://www.sportvue.com/>



Microsoft HoloLens 2
<https://www.microsoft.com/it-it/hololens>

Computer Science vs Web Science

Metrics

CS

Moore's Law

Order (n) algor.

Gigabytes

WS

page visits

visitor/month

songs/video

CS vs WS:

Subjects

CS

Computer networks

Packet exchange

Information

Programming languages

Databases, operating systems

Compilers

3D Graphics, rendering algos

Computational geometry

WS

social networks

voice over IP, musical sharing

relationships

Wikis, blogs, tags, fotologs

E-government, E-learning

Medical Info medica, business

Map creation and sharing

Animation, music, foto, videoclips

CS vs WS:

Focus

CS

Technology

Computers

Supercomputers

Efficient programming

WS

Applications

Users

Mobile devices

Universal Usability

approaches to design

- User interface design requires the cooperation of computer scientists and psychologists for a good understanding of algorithms and programs (computer side) and people (user side)
- two approaches arise:
 - 1) where computer scientists want to have a well designed interface after a first shot - using formal design rules - and
 - 2) where psychologists would like an iterative design in order to gradually improve the developed system.

incompatibility

- when computers cannot be interfaced, we say they are "incompatible"
- when humans and computers cannot communicate we never refer to incompatibility although this is the case
- do interactive programs aim at special user features?
- better design: user centered - even user co-author
- ...to switch off Windows you have to choose the start-up icon!

observations

- treat people at least as well as we already treat computers
- is too easy to ignore important design issues
- designers tend to overcome bugs by adding ad-hoc features, extending the systems and making them more complex
- conflicts and limitations in the original design can be camouflaged with local add-ons
- new features are unrelated, may have side-effects and no accurate forecast of what may happen to other actions performed through the new interface
- typically, designs grow bottom-up by accruing features

Composition Fallacy

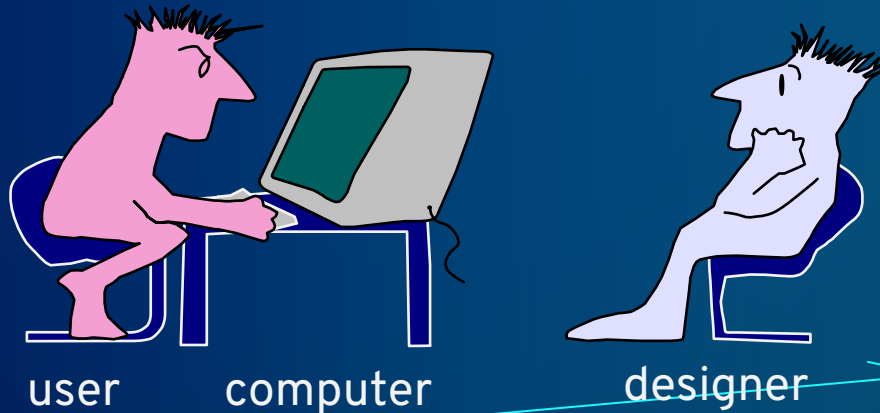
- properties of the parts are also properties of the whole
(composition)
- a **good** interface may have **bad** parts
- good parts may exist within a bad interface

abstraction

- user is the participant with a choice - any kind of participant - naive, expert, discretionary, disabled, etc.
- computer is the participant with a program - it obeys instructions
- designer the participant who anticipates the possible choices of the user encoding them into a computer program
- pragmatically, many different persons participate to the design of computer systems: manager, programmer, analyst, technical author, evaluator and user

interaction

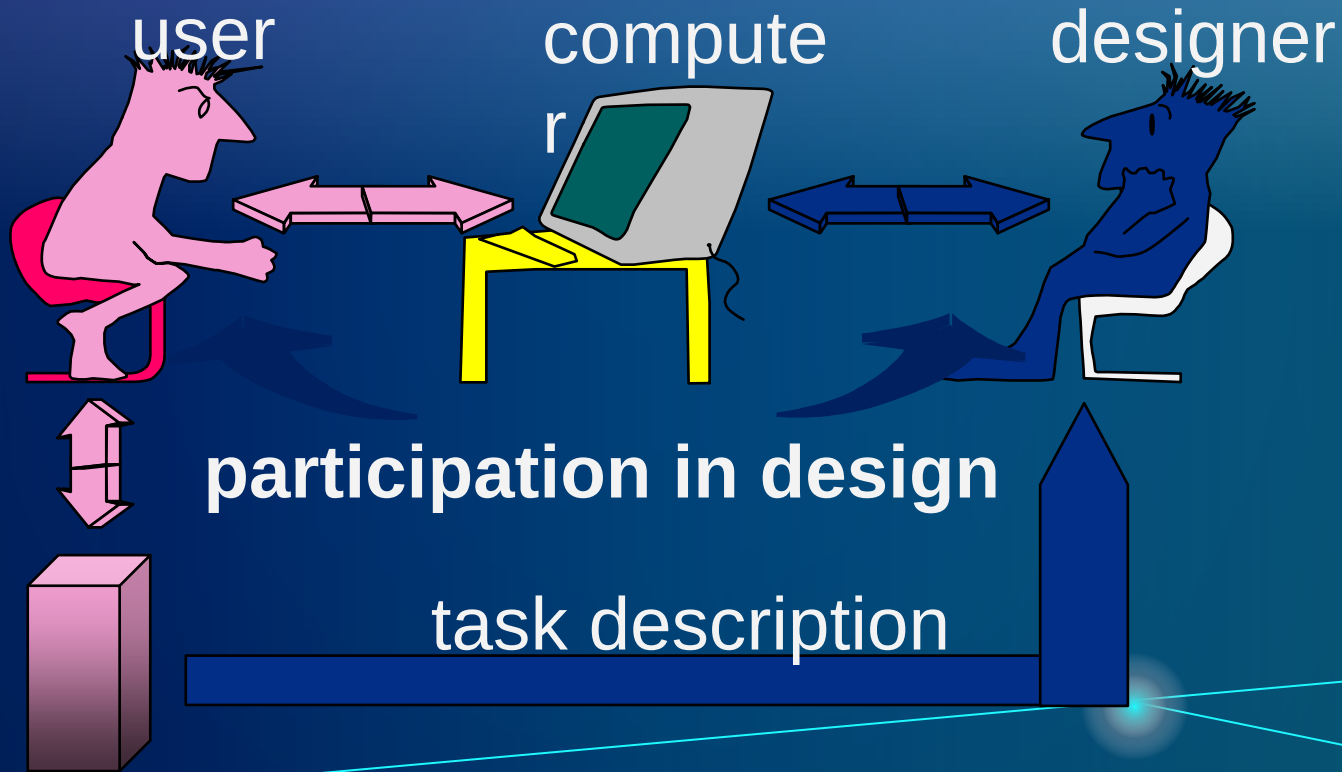
- humans understand through modelling
- computer users build up their own models of the systems they will be using
- constant activity helps to achieve tasks even if not all the details are known provided there are no side effects



communication bandwidth



design information



mental models & programs

- additional information comes from
 - on the user's side - mental models and
 - on the computer's side - programs but...
- mental models and programs are not equivalent!
- they are not isomorphic
- not synchronized
- use different logical systems
- have different bugs and limitations.

programs and mental models

- if they were equivalent, one of them would be sufficient
- either the mental model or the program
- the program may be run and turns useful if, in some way, it **matches** the mental model
- every computer system has been built on the basis of explicit or implicit models of the use
- of his/her tasks/needs/expectations
- generally a canonical (according to a formula) model of the user is employed by the designer

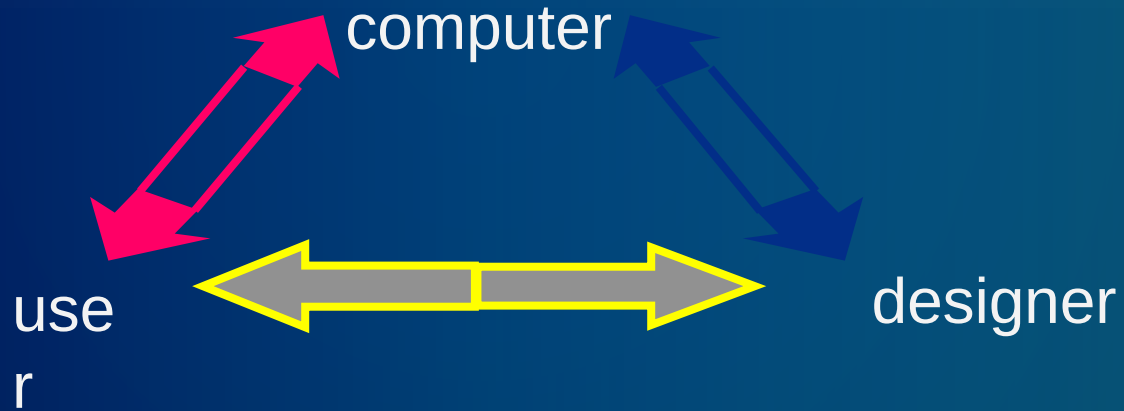
improvements

HC-interaction may be improved by

- increasing the **coherence** between the mental model and the canonical model
- decreasing the **reliance** on that coherence
- increasing the **information capacity** of the interface

bottlenecks

- information **transfer** bottlenecks (I/O)
- information **processing** bottlenecks (mental)



non-determinism

- **randomness** may be requested but generally is a non-wanted situation: bugs - they come from unexpected external situations, including the user
- computer programs should be predictable, if the user has no adequate model for a situation, it may appear random
- a particular interference from the real world is the one of time, real-time races are performed whenever decisions are made depending on which information arrives first
- a schedule for erasing old files should be known otherwise those files exceeding the time threshold will be removed and this may appear random to the user
- non-determinism may appear just because some information is not known
 - note-takers, diaries, transcripts help to "keep track"

user

- difficult to maintain consistent beliefs - along time
- not using an adequate notation
- typically wanting transitive preferences (A better than B better than C/ implies A better than C)
- menu interface better than command-based one
- knowledge-based direct manipulation better than menu

short-term memory

- selective attention, shifting attention = 50 ms, eye movements 200 ms, audio is captured quickly and faster than video
- once selected, the information is copied in STM: a conscious place which may manage 7 objects, if unused the contents of STM decays after 20" - it can be refreshed by rehearsal
- data is stored in chunks each one being a name related to an object, the chunks (output chunks are routines) may contain sub-chunks which may be stored and processed
- chunks are built by closure, STM may be seen as a memory stack, the closure is a pop operation

on closure

- closure may eclipse other goals - this may cause a termination error caused by a 'too effective closure'
- in a cash dispenser if the money is provided AFTER the card has been returned, the user will never forget his card in the machine (termination error)
- resource limited: too many processes to be looked at are happening
- data limited: darkness for eyesight
- these features are important in the design of warning/help clues within interfaces

long-term memory

- > 5" the contents of STM gets transferred in LTM having a **large** capacity and **small** decay
- our knowledge on language is contained in LTM: it is 'perfect' only sometimes it is difficult to **retrieve** - particularly without rehearsal - whenever successful it takes .1 "
- mnemonics helps to retrieve wanted data, previous rehearsal in STM also helps: I could insert my table on ≠ behaviours of people and machines during interaction
- people can never forget, computers may completely erase data and sometimes users forget this...

l-tm & s-tm

- both suffer from interference since different items in memory may have the same coding (as if Hash coded) - it becomes hard to recall one thing since it was recorded with two different codes, or the same code for two different items
- for 2 different interactive systems, the experience of the first, carries-over the learning process of the second if they are similar - as when programming with a new language/speaking a new language in terms of an older, known one (conceptual capture)

on behaviour

- example of priming: impinging an idea into someone - spell "shop"- what do you do when you see a green light? (go and not stop)
- stereotyped behaviour is when people panic and use a "default escape" solution that generally is not the best action to perform under the circumstances - people may make mistakes in this way and, even worse, are not able to recover from such mistakes...

cognitive dissonance

- *Festinger, 1957*
- after a (program) choice is made, the new alternative looks worse than the other, possible one, and...vice versa.
- humans try to reduce such dissonance to feel better
 - e.g. the user may change his program, reduce the importance of choice (his job is more significant than the program he is using), add further consonant ideas, selectively expose himself to read positive evaluations of his choice...(tricks that help)

users adapt

- it is easier to stay with nonsense, than face the cost of changing over to something which would almost certainly be better
- a user will adapt, and then the designer will continue with the same choice he made in the previous design